

Especificación

Proyecto Final programado a desarrollar para su defensa grupal en Semana 15.

Los grupos deben estar conformados por 3 estudiantes y un máximo de 4. Cada integrante del grupo deberá, **de forma individual**, subir al aula virtual los recursos solicitados para cada entrega.

Los problemas entre compañeros del grupo deben solucionarse a lo interno de cada grupo. En caso de ser necesario, notificar vía correo electrónico al profesor.

Para cada entrega se deberá subir **un solo archivo** en formato .zip o .rar con los 2 elementos detallados más adelante: Readme.md y el proyecto de código, podrá facilitar como “recurso” o evidencia de su trabajo el enlace al proyecto de GitHub de su proyecto con los dos elementos mencionados anteriormente.

Entregables

El código y demás archivos requeridos, debe entregarlos durante la clase en la semana pactada para cada entrega, después de esto no se recibirán y se perderá la calificación de esta evaluación.

Cada integrante del grupo deberá subir el entregable al campus virtual, en un único archivo .zip ó .rar, como se indicó anteriormente, o el enlace/URL de GitHub en su efecto. Se aclara que debe adjuntarse todo el **proyecto** exportado por **NetBeans** (no se reciben clases sueltas).

En la carpeta también debe agregar un archivo de texto con el nombre **Readme.md** donde indique:

1. Número de grupo asignado
2. Integrantes del grupo. **Solo los que participaron en la entrega.**
3. Los requerimientos implementados en esta entrega. Aunque estén incompletos.
4. Los requerimientos pendientes **no** alcanzados en esta entrega.
5. Detalles de configuración, credenciales u otra característica que sea necesario para correr la aplicación.
6. Documentación breve de clases y métodos.

Se evaluarán los temas según hayan sido vistos en clase. Si cumple con los requisitos especificados se asignan todos los puntos del tema, sino se descuentan según se incumpla y la rúbrica previamente entregada.

No está permitido ningún *framework* o código previo. Todo código debe ser generado por los estudiantes desde cero. El código de cada grupo debe ser desarrollado por cada grupo por separado.

Desarrollará un proyecto utilizando la clase JOptionPane de la librería SWING de Java para la interfaz. Si se quiere cambiar a otra interfaz debe justificarse con el profesor previo a las entregas.

Las clases y los métodos deben tener documentación mediante comentarios en el mismo código.

Caso

La empresa **BusNova** es una empresa que se encarga de brindar soluciones de TI para diferentes empresas, actualmente tiene un proyecto el cual desean encargarlo a su equipo de programación. Este proyecto consta de una herramienta tecnológica que permite la gestión de **buses** y sus servicios llamada **BusNovaTech**. La empresa cuenta con múltiples terminales y servicios en todo el “universo”.

La información de transacciones debe ser persistente y debe poderse ver el histórico de los trámites y servicios brindados (base de datos de archivos JSON).

De igual forma, el sistema debe contar con un login de al menos dos usuarios, sin esto no le permite acceder a las funciones del sistema. El sistema debe contar con al menos los siguientes módulos:

Módulo 1.0: Configuración	Estructuras de Datos
Este módulo cumple con la finalidad de configurar el programa.	- Utilizará como medio de almacenamiento persistente un archivo config.json. Este mecanismo será su base de datos. - Este módulo solo se ejecuta la primera vez que el programa entra en ejecución posterior a esto el programa carga la configuración dada por el usuario almacenada en el config.json. Si este archivo se borra, deberá volver a ingresar la información. - Este módulo permite ingresar el nombre de la terminal, así como la cantidad de buses para la atención de clientes. Entre los buses siempre debe contar con un único bus preferencial, la cual atiende a personas con condiciones preferenciales como discapacidades, embarazo, empresariales, adultos mayores. De igual forma, entre los buses siempre debe contar con un único bus directo para viajes rápidos no preferenciales (una sola parada) - Los demás buses son normales aplican para dos o más paradas no preferenciales. - El número total de buses puede cambiar por cada ejecución no debe ser configurado de forma fija dentro del código. Debe poder ser dinámico sin afectar la funcionalidad del programa (lista de colas). - Lo anterior debe verse reflejado (serializado) en el archivo config.json . - Los usuarios y contraseñas de los usuarios serán almacenados en este archivo. No deben quedar quemados en el código (ideal un usuario por miembro del grupo).

Módulo 1.1: Creación de tickets	Estructuras de Datos
Este módulo contempla la creación de los tickets	- Utilizará como medio de almacenamiento las Colas de Prioridades según el tipo y el tamaño de la Cola. - El nodo de la cola debe contemplar como mínimo la siguiente información para cada ticket: - Nombre - Id - Edad - Moneda Cuenta (dólares ó colones) - Hora de compra (la obtiene del sistema al generar el ticket) - Hora de abordaje (por defecto es -1 hasta que sea atendido) - Servicio (VIP, Regular, Carga, Ejecutivo, por defecto es “NA”) - Tipo Bus (P: Preferencial, D: Directo, N: Normal parando) importante el ticket solo puede tener un tipo. - Deberá contemplar cualquier otro atributo dentro del Nodo para que este funcione como una cola. - Una vez el ticket es creado con la información del cliente, espera su turno para abordar el bus que le corresponde. - Lo anterior debe verse reflejado (serializado) en el archivo tickets.json. Y cargarse

cuando inicia el sistema si existen registros pendientes de atender.

Módulo 1.2: Atención de tiquetes

Estructuras de Datos

Este módulo cumple con la atención de tiquetes

La atención de tiquetes se divide en dos funciones:

1. La primera es al momento de crear un tiquete si se asigna a una cola vacía, inmediatamente este pasará a ser atendido por el inspector del bus, siempre y cuando este esté desocupado (no se encuentra atendiendo un tiquete).
2. La segunda función es cuando las filas se encuentran llenas, en este escenario, el programa debe contar con una funcionalidad que sea "Abordar", con la cual el tiquete es atendido por el inspector. Una vez atendido debe poderse marcar el tiquete como "Atendido" y dejar libre al inspector para la siguiente persona.

Cada vez que un tiquete pasa a ser atendido debe cambiar la hora de atención por la del sistema y además debe mandarlo a guardar en la base de datos, en este caso un archivo llamado atendidos.json

Se debe guardar en base de datos en cual **bus** se subió y de cual **terminal** compro el tiquete.

El sistema debe poder mostrar por pantalla esta información con una opción de menú.

Cada bus tiene un inspector y su función permitir la entrada al bus. Si cambia a estado "Atendido" es porque ya subió al bus.

Los tipos de servicio de los tiquetes son al menos:

- VIP: paga 100\$ adicionales, con derecho a comer y beber.
- Regular: cobro normal de 20\$.
- Carga: paga 20\$ más 10\$ por cada lb de carga.
- Ejecutivo: paga 1000\$ adicionales, con derecho a manejar el **bus**.

Los servicios se cobran al subir al bus por el inspector, se debe preguntar al usuario, si se niega a pagar es sacado de la fila de ese bus y tiene que iniciar el proceso de atención de nuevo.

El cálculo requiere el consumo en línea del web service del Banco Central.

Módulo 1.3: Llenado de las colas

Estructuras de Datos

Este módulo contempla las técnicas de llenado de las colas de los buses

Los buses se llenarán conforme las personas soliciten tiquetes, además, debe considerar el tamaño actual de la cola para el bus, es decir:

Bus 1 | tiene 5 personas en fila

Bus 5 | tiene 3 personas en fila

Bus 7 | tiene 4 personas en fila

En caso de un tiquete normal nuevo, se asignaría a la fila del bus 5 porque tiene menos personas en espera a subir, en caso de empate, se puede asignar a cualquiera.

Cabe resaltar que para los tiquetes de Bus Directo o Bus Preferencial serán asignados a conforme se crean ya que estos tiquetes no se atienden por otros buses (solo hay uno directo y uno preferencial).

Lo anterior debe verse reflejado (serializado) en el archivo colas.json. Y cargarse cuando inicia el sistema si existen registros pendientes de atender.

Módulo 1.4: Servicios Complementarios	Estructuras de Datos
Este módulo contempla las técnicas de búsqueda en grafos	Mediante la implementación de grafos se deberá definir productos complementarios según la gestión del cliente: • Definir un grafo ponderado dirigido. • Definir diferentes localidades donde el bus pararía. • Imprimir el grafo construido con las localidades y peso de las rutas. • Permitir al usuario saber la ruta más corta entre dos terminales que el elija. La información se presentará como una opción de menú y no se le cobrará al cliente. Lo anterior debe verse reflejado (serializado) en el archivo grafo.json. Y cargarse cuando inicia el sistema.

Módulo 1.5: Consulta BCCR	Estructuras de Datos
Este módulo contempla la consulta de web services al BCCR para conocer el tipo de cambio del dolar.	El sistema debe permitir en una de las opciones conocer el tipo de cambio del dólar en línea, al día de la presentación, por esta razón, dicha consulta debe ser implementada utilizando el Web Service del Banco Central de Costa Rica. Ver: https://www.bccr.fi.cr/indicadores-economicos/servicio-web/gu%C3%ADa-de-uso https://www.bccr.fi.cr/indicadores-economicos/servicio-web

Persistencia de la información

1. La información se guarda en archivos JSON(.json), no cifrados, como mecanismo de simulación de base de datos. Si alguno de los archivos no existe y se requiere la información el sistema debe estar en la capacidad de solicitarla para funcionar correctamente.
2. Si el sistema se apaga y existen clientes en las filas de los buses, al levantar el sistema los clientes deben cargarse en sus lugares respectivos, como si el programa nunca se hubiera cerrado (cargando desde los archivos JSON).

Presentación y Demostración de la implementación.

La presentación y demostración se hará a la semana de la entrega del código. Deberá durar 10-15 minutos en total. La exposición se hará escogiendo el orden de los grupos que pasarán al frente según su número asignado en un sorteo aleatorio. Uno tras otro hasta terminar. El uso de PowerPoint es opcional, pero altamente recomendado. El estudiante seleccionado deberá presentar:

1. Introducción:
 - a. Introducir a todos los integrantes del grupo
 - b. Presentar la temática del proyecto
2. Demostración:
 - a. **Realizar la prueba de cada módulo funcionalmente en vivo.** Manejando la interfaz con el usuario y mostrando ejemplos específicos que demuestren el cumplimiento de todos los requerimientos.
 - b. Verificación de la persistencia en los archivos (serialización y deserialización de objetos).
 - c. Cerrar y abrir el sistema para verificar la carga y actualización del sistema.
 - d. Tolerancia a fallos del sistema o manejo de errores.
3. Implementación:
 - a. Presentar el proyecto, cómo implementan sus algoritmos a profundidad y la documentación realizada en el código para demostrar el cumplimiento de los temas y sus condiciones. Este punto por tiempo se abarca en el vídeo adjunto en cada entrega. El objetivo es que cada participante explique la parte que desarrollo del código y la forma de solucionar los retos planteados.
4. Retos y lecciones:
 - a. Compartir con la clase los retos y lecciones de realizar la visión de las funcionalidades de su proyecto en cuanto a decisiones de diseño, discusiones de implementación, entre otros.
 - b. Indicar si existieron partes del proyecto que no pudieron implementarse.

Notas Adicionales

- Se realizarán 3 avances del proyecto con el fin de asegurar la calidad de este, las mismas están estipuladas en el plan del curso.
- No se solicita trabajo escrito, pero su se debe documentar el código.
- En cada uno de los tres entregables se debe indicar en el archivo Readme.md cuales son los módulos implementados y cuales correcciones se realizaron de las entregas anteriores.
- Se recomienda acatar los comentarios y apuntes realizados por el profesor en cada entrega.
- Si tiene un problema con un compañero de grupo, comuníquemelo lo antes posible.
- La asistencia al día de la exposición es obligatoria. No se puede reponer, solo se puede justificar la ausencia.
- Cada estudiante deberá mostrar dominio completo de la aplicación desarrollada.
- El archivo .zip o url de GitHub del proyecto debe ser subido al aula virtual por cada estudiante **de forma individual**. El estudiante debe verificar que se descargue y se pueda descomprimir correctamente desde el aula virtual. En caso de presentarse errores, estar corrupto el archivo, o tener contraseñas para descomprimir, se pierde la entrega.

- No puede retirarse de la clase hasta que expongan todos los grupos.
- Si tiene un problema con un compañero de grupo, debe documentarlo por correo al profesor, con copia a todos los miembros del grupo.
- Debe respetar los algoritmos vistos en clase. El uso de estructuras precompiladas de cualquier tipo: vectores, ArrayList, List, Map, HashMap de la librería java.utils o similares, **resultará con la pérdida completa de los puntos** según el ítem a evaluar. Solo puede aplicar las soluciones vistas en clase. Si puede aplicar herencia y extender el código visto en clase.
- Si se comprueba la intervención de personas ajenas al grupo en la programación del proyecto, éste será anulado y **será causa para la pérdida total de los puntos**. Se aplicará la misma sanción en caso de que se compruebe que existe copia parcial o total de códigos de Internet o inteligencias artificiales. Dicho caso se elevaría a las instancias universitarias correspondientes.

Rubrica de evaluación

Según los contenidos del plan del curso, leído y aprobado en la semana # 1.